

Computer Architecture Lab

(EL-2012)

LABORATORY MANUAL

Fall 2026



(Lab#01)

Apply modern simulation tools ModelSim and Logisim to implement basic processor components

Student Name: _____ Roll No: _____ Section: __

Date performed: _____, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Engr. Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Learning Objectives:

To get introduced to:

- 1- ModelSim
- 2- Hardware Description Language
- 3- Build and simulate digital circuits using Logisim and Verilog
- 4- Build small modules for logic gates, and design circuits using gates

Tool required:

ModelSim 10.5b (32-bit)
ModelSim

Hardware Description Language (HDL)

Hardware description Languages including VHDL, Verilog, System C and Handle-C are frequently used for FPGA programming. VHDL and Verilog are matured as industry standards.

Verilog Levels of abstraction:

1. Switch (Transistor) Level

The lowest level of abstraction for a digital HDL would be the switch level, which refers to the ability to describe the circuit as a net list of transistor switches.

2. Gate (Structural) Level Modeling:

A slightly higher level of abstraction would be the gate level, which refers to the ability to describe the circuit as a net list of primitive logic gates and functions. The gates have one scalar output and multiple scalar inputs.

Gate Type	Description	Instantiation Syntax
and	N-input AND gate	and a1(out,in1,in2) ;
nand	N-input NAND gate	nand a2(out,in1,in2) ;
or	N-input OR gate	or a3(out,in1,in2) ;
nor	N-input NOR gate	nor a4(out,in1,in2) ;
xor	N-input XOR gate	xor x1(out,in1,in2) ;
xnor	N-input XNOR gate	xnor x2(out,in1,in2) ;
not	1-input NOT gate	not g1 (out , in) ;

Fig 2: Verilog Structural modeling operators

3. Data Flow Level Modeling:

For small circuits, the gate level modeling approach works very well because the number of gates is limited, and designers can instantiate and connect every gate individually. However, in complex design the number of gates is very large. Thus implementing the function at a level higher than gate level is a good choice. Dataflow modeling has become a popular design approach as logic synthesis tools have become sophisticated. This approach allows the designer to concentrate on optimizing the circuit in terms of data flow.

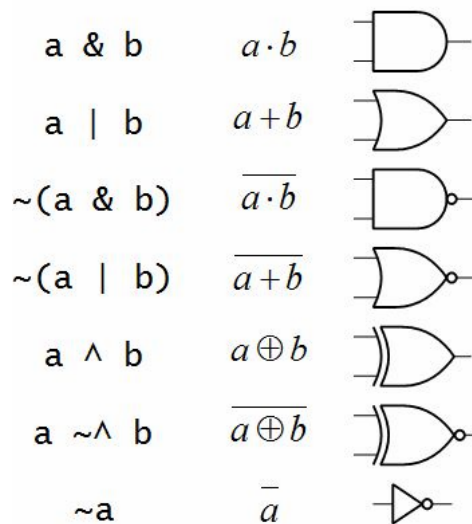


Fig 3: Verilog Data Flow Operators

4. Behavioral level Modeling:

Behavioral modeling in Verilog is used to describe the function of a design in an algorithmic manner. We will cover details of behavioral modeling in upcoming Labs.

Source code examples:

1. Gate level modeling:

```
module gates (a,b,c); // module name and variables to be used in program
input a,b;           //define inputs
output c;            //define output
or(c,a,b);           //operation
endmodule            //end of program
```

2. Data flow modeling:

```
module gates (a,b,c); // module name and variables to be used in program
input a,b;           //define inputs
output c;            //define output
assign c= a | b;      //operation
endmodule            //end of program
```

Introduction to ModelSim

ModelSim is an easy-to-use yet versatile VHDL/(System)Verilog/SystemC simulator by Mentor Graphics. It supports behavioral, register transfer level, and gate-level modeling. ModelSim supports all platforms (i.e. Linux, Solaris and Windows) and many others too. On Linux and Solaris platforms ModelSim can be found preinstalled (see [Linux/Mustatikli](#)). Windows users, however, must install it by themselves.

Installation of ModelSim

Download ModelSim from official Intel's website:

<https://www.intel.com/content/www/us/en/software-kit/750368/modelsim-intel-fpgas-standard-edition-software-version-18-1.html>

Downloads

Linux Software

Windows Software

ModelSim Software

ModelSim-Intel® FPGA Edition (includes Starter Edition)

Download
ModelSimSetup-18.1.0.625-windows.exe

After selecting Windows Software of ModelSim, Accept the Software License to download.



Software License Agreement

Legal Disclaimer

PLEASE NOTE: This version of software ("Software") does not contain the latest functional and security updates. In order to use this version, you must first acknowledge the following term, which supplements and supersedes any inconsistent provision in the version of the Intel® FPGA Software License Subscription Agreement for the product (e.g., Intel® Quartus® Prime Software, Intel® HLS Compiler, Intel® FPGA SDK for OpenCL™, DSP Builder for Intel® FPGAs, or Advanced Link Analyzer) with which you use the Software:

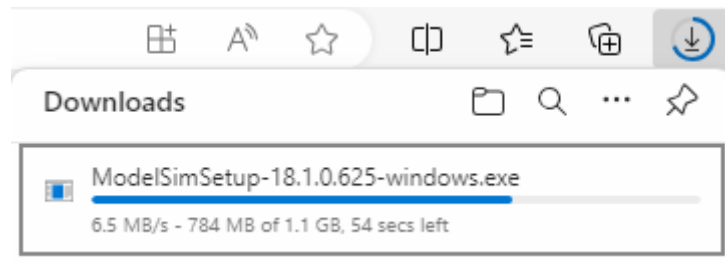
Intel does not give or enter into any condition, warranty, or other term:

- i. with respect to any malfunctions or other errors in its Software caused by virus, infection, worm or similar malicious code not developed or introduced by Intel; or
- ii. to the effect that any Software will protect against all possible security threats, including intentional misconduct by third parties. Intel is not liable for any downtime or service interruption, for any lost or stolen data or systems, or for any other damages arising out of or relating to any such actions or intrusions or resulting from use of Software. Intel does not give or enter into any condition, warranty, or other term with respect to interoperability.

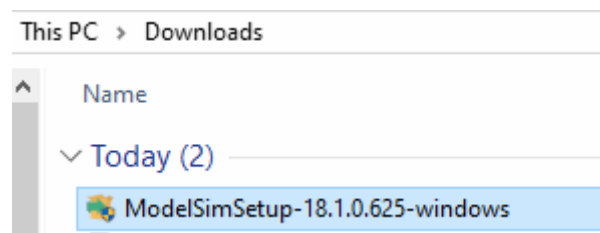
Intel does not warrant or assume responsibility for the accuracy or completeness of any information, text, graphics, links or other items within the Software. Please click "Accept" below to continue the download process.

Accept

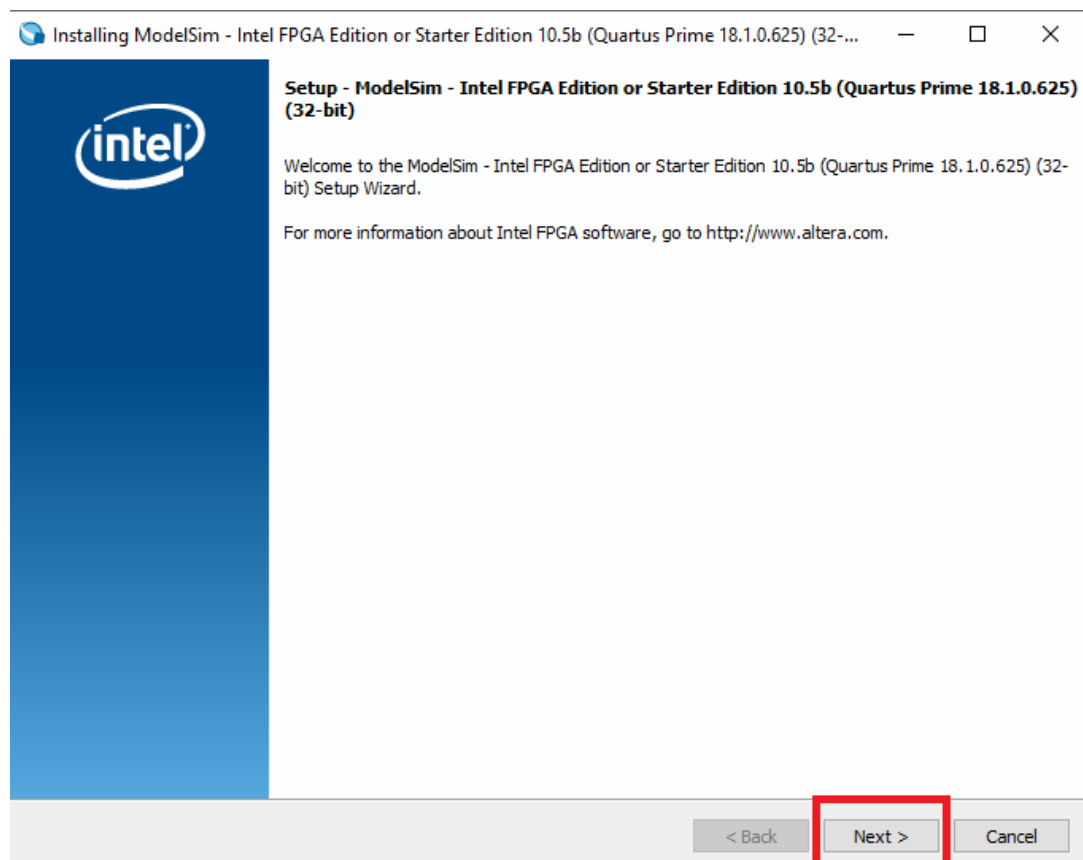
Reject



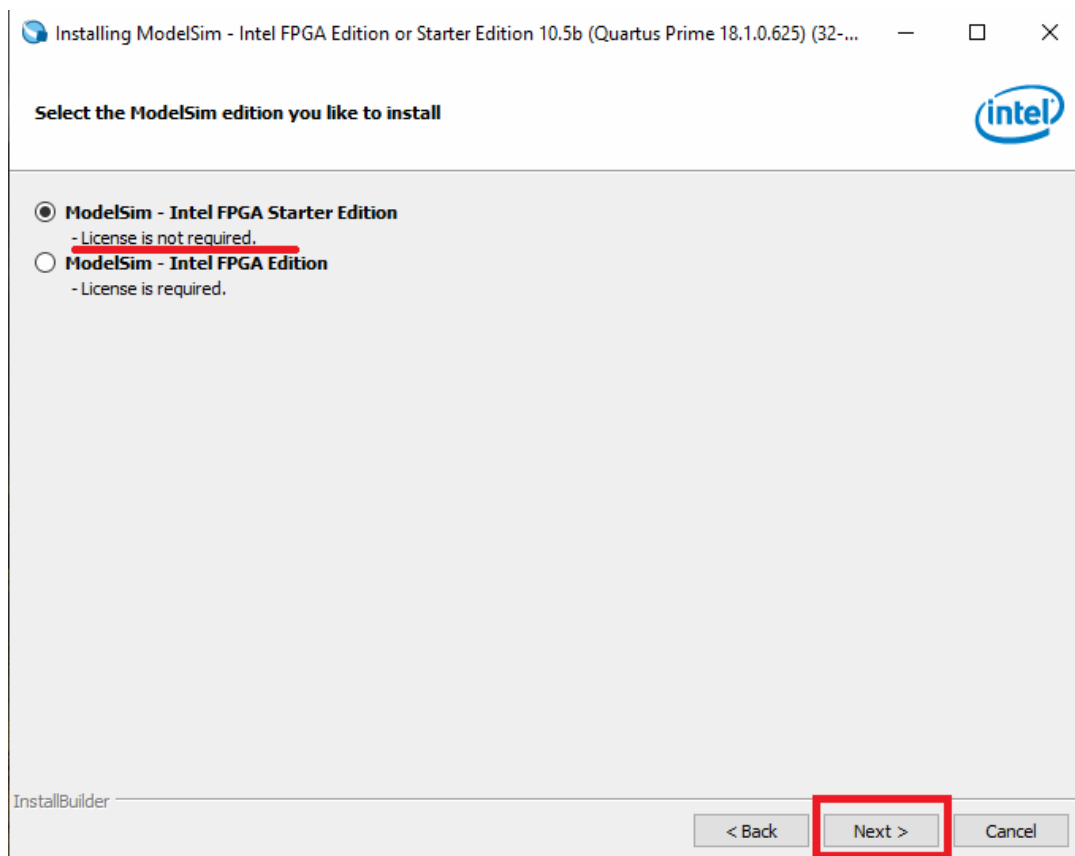
After download, open the software's setup on your local machine.



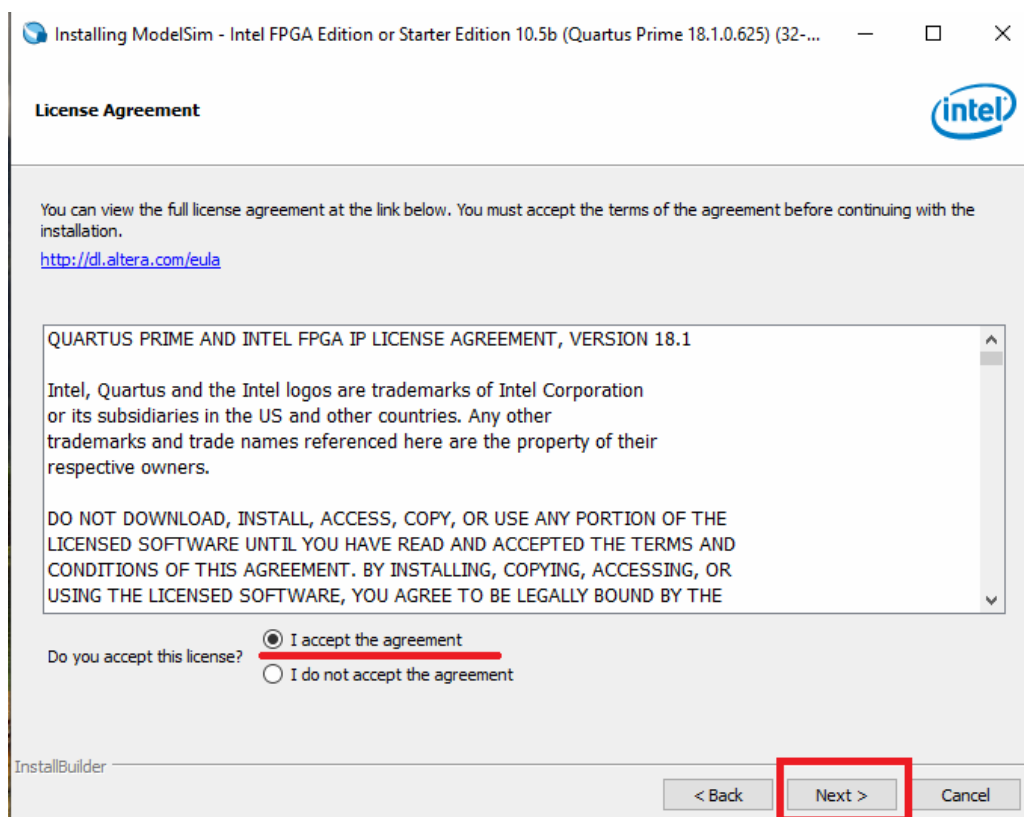
When the welcome window of ModelSim's software, click next button



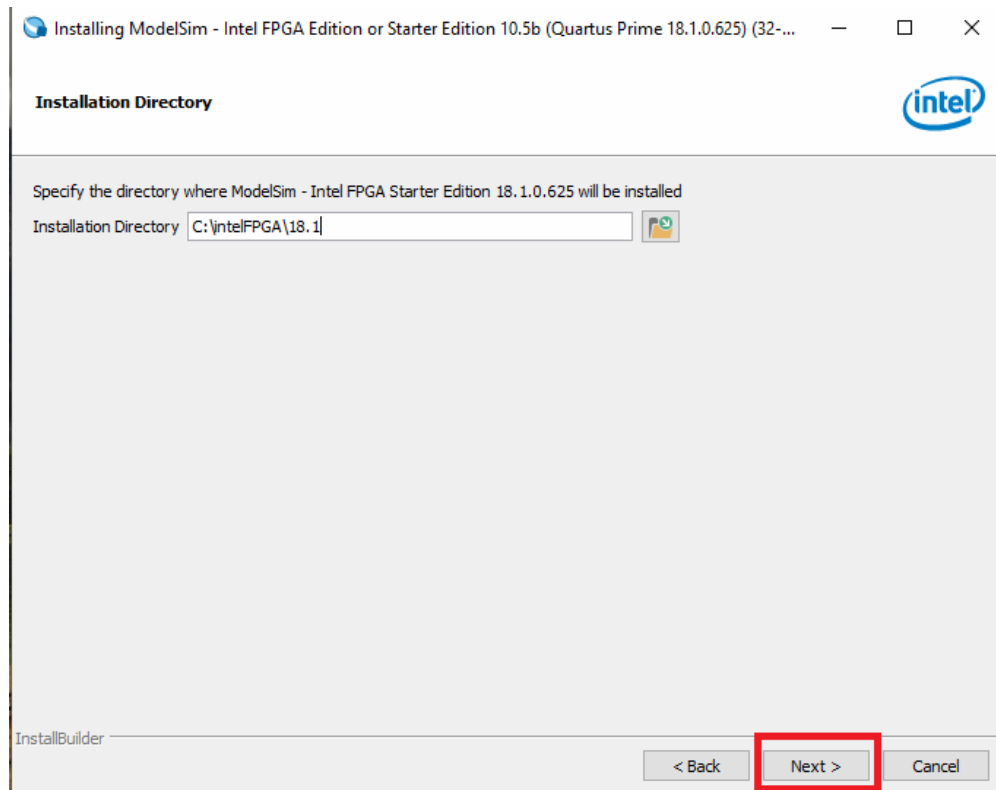
Select the free licensed free software and click the next button.



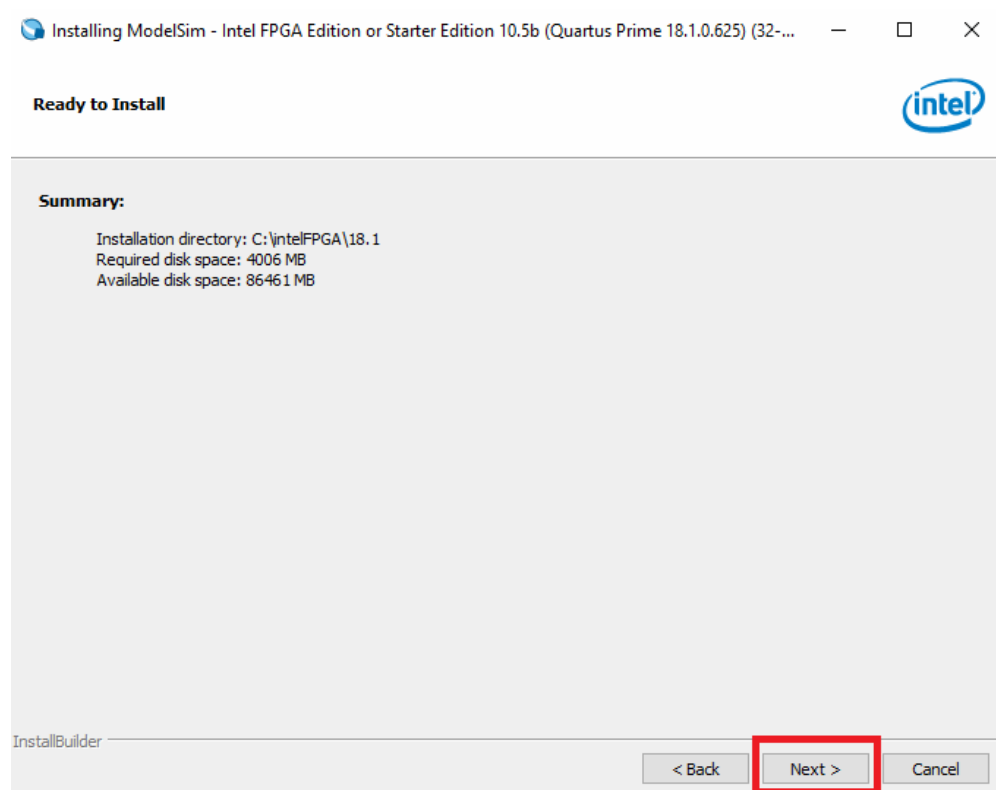
Accept the free License agreement and click the next button.



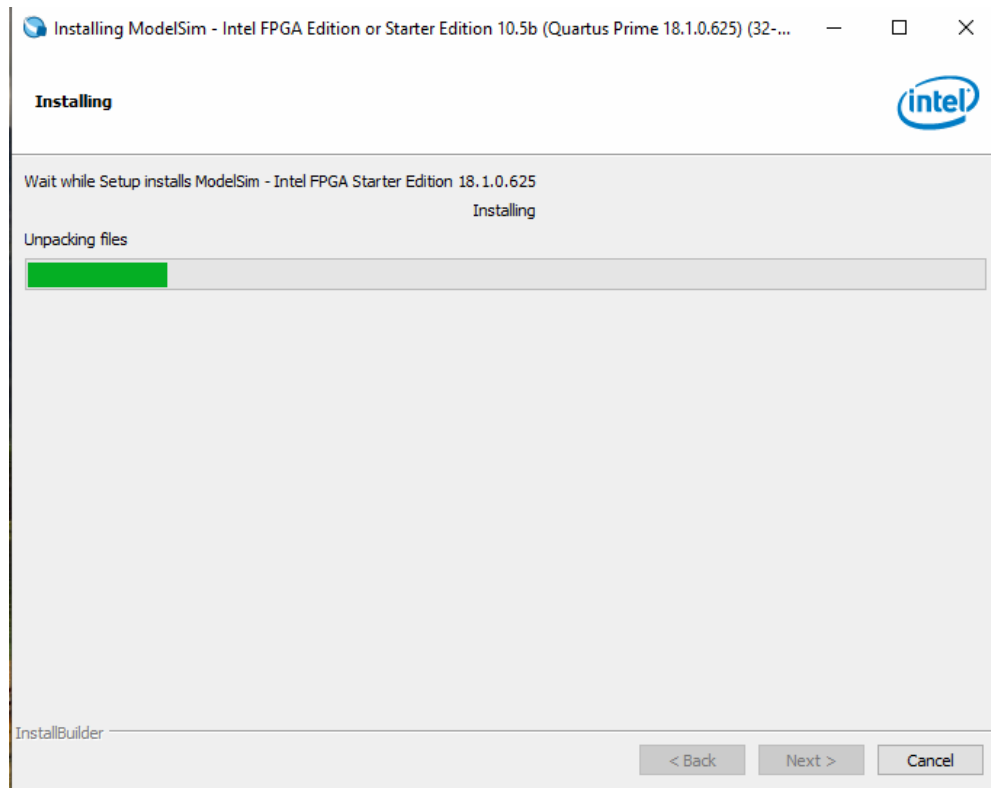
Insert the directory of your local machine where you want to install & click the next button.



Click the next button, if your directory contains enough space for installation.



Wait for installation.

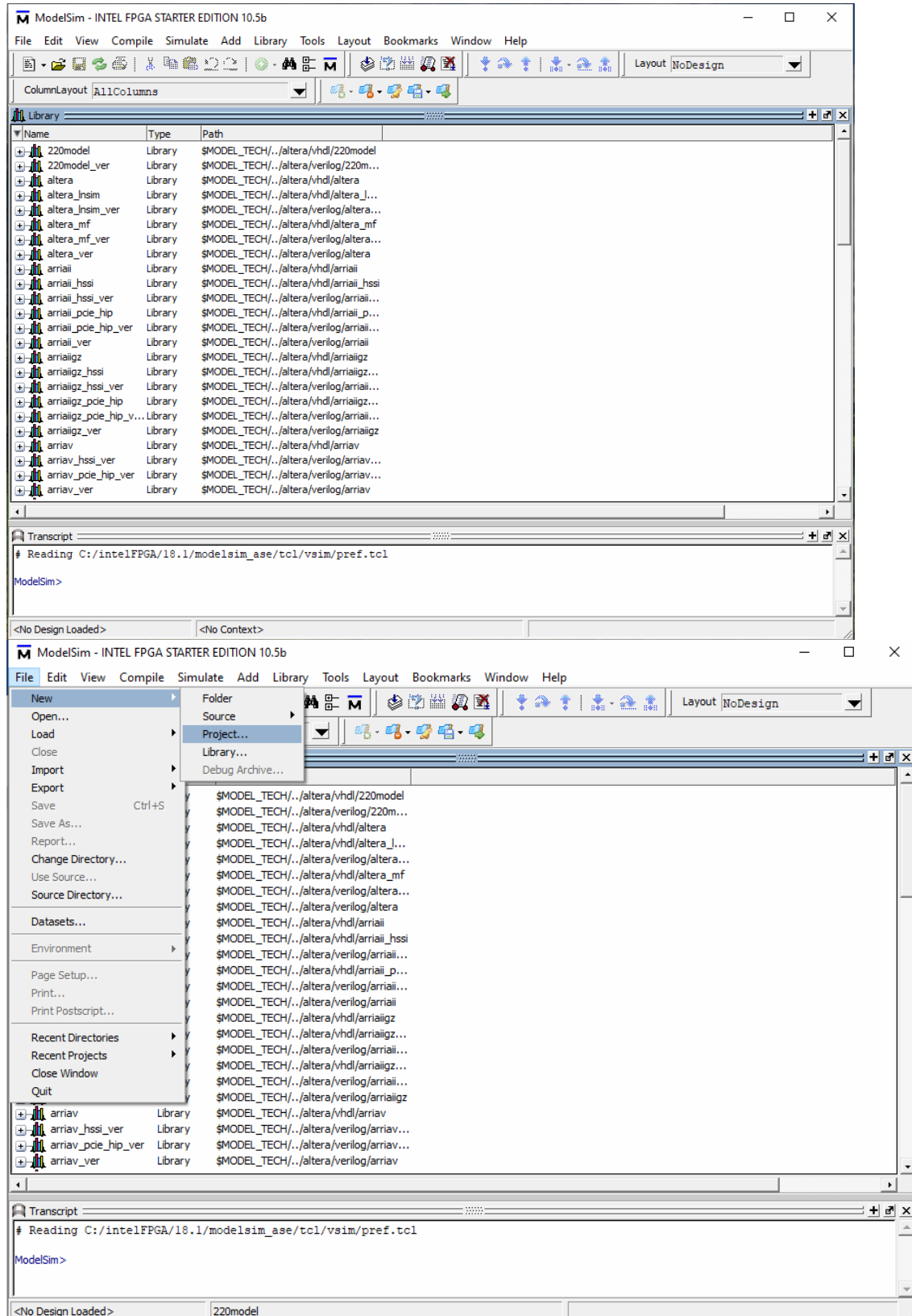


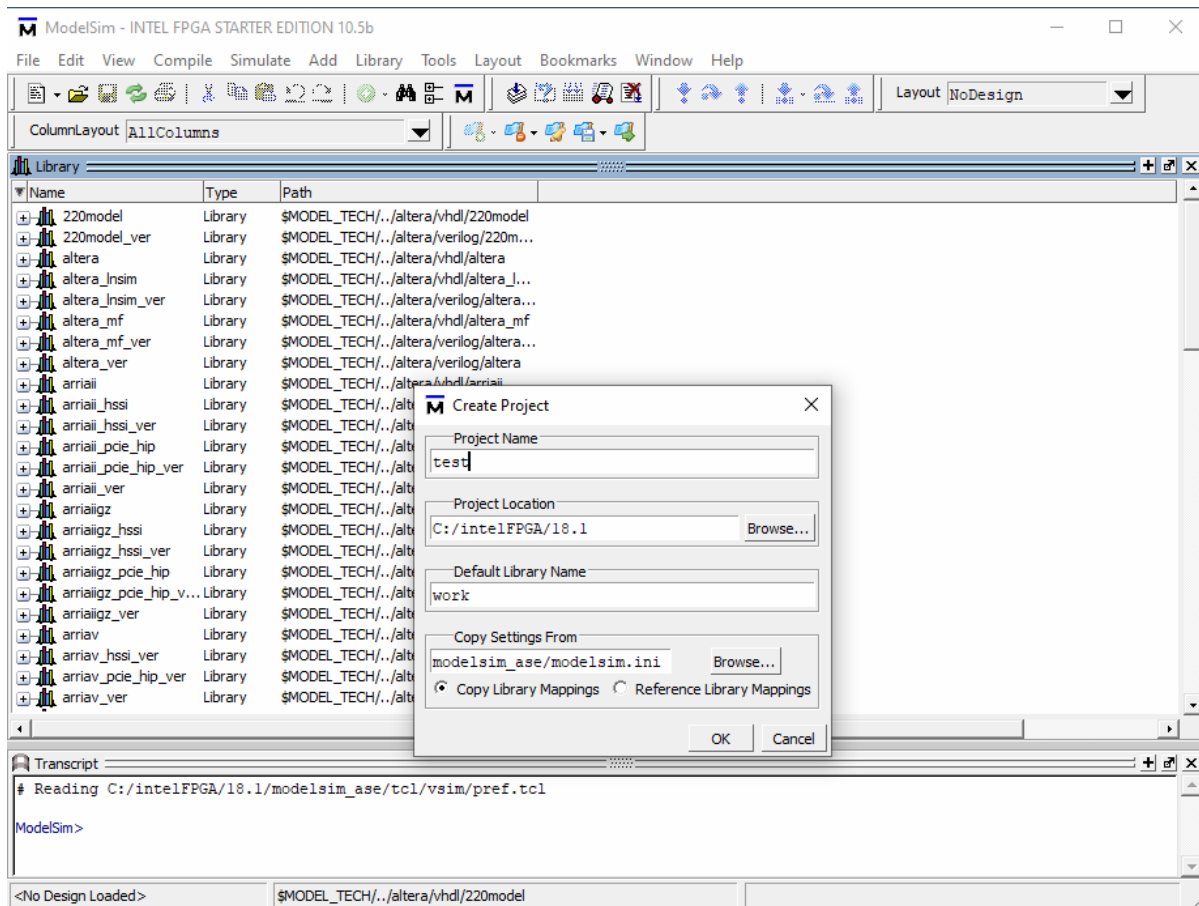
Click the finish button, to complete the installation process.

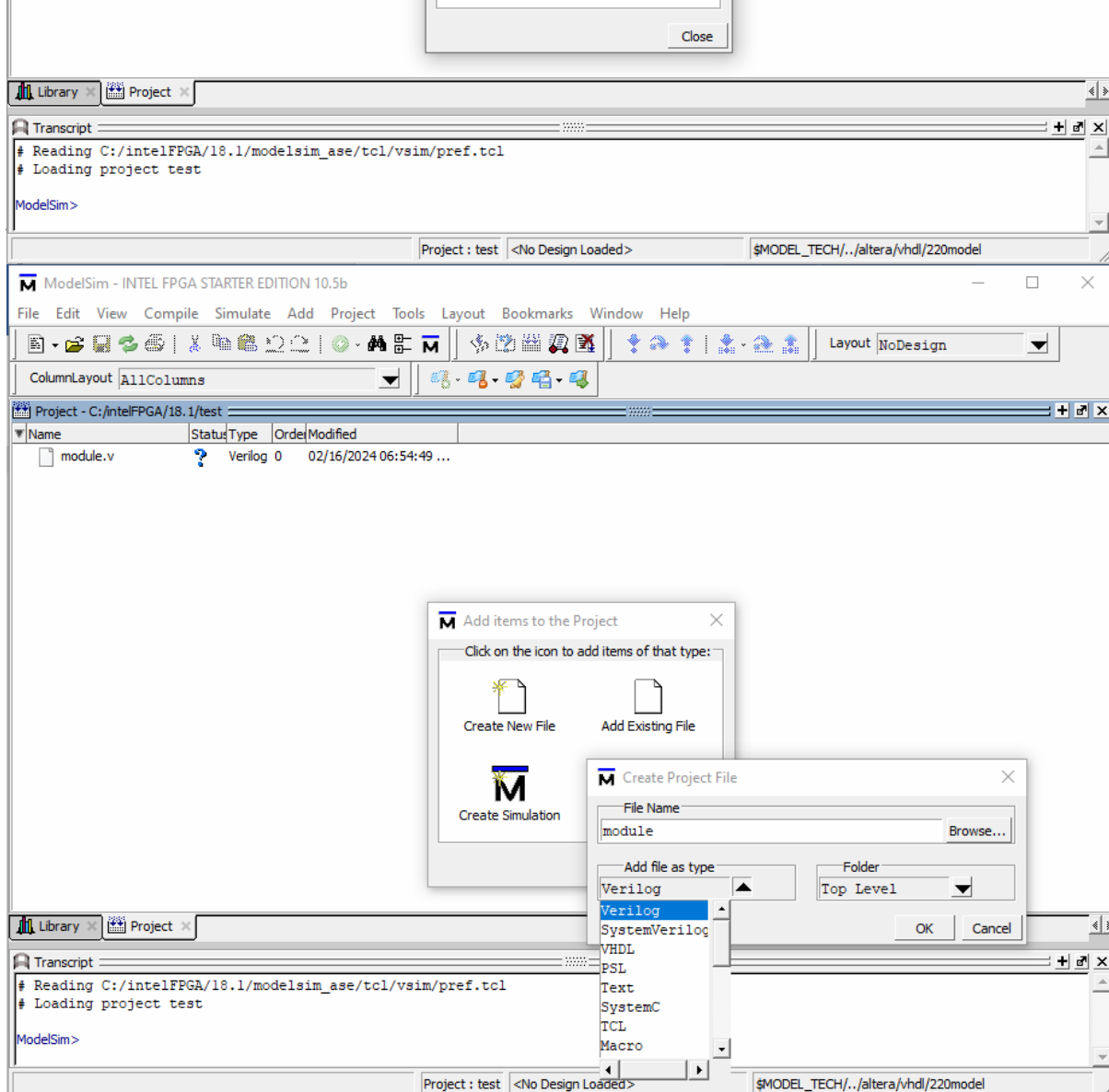


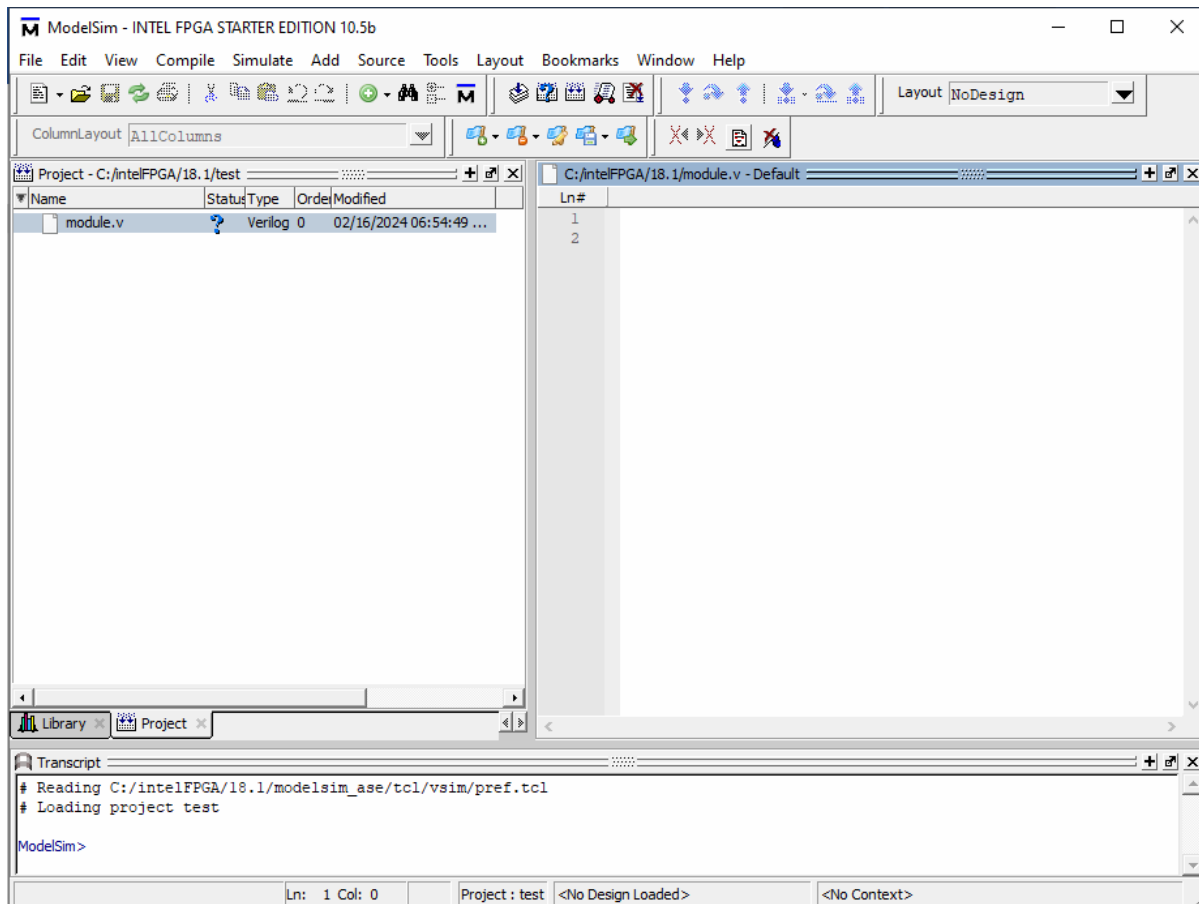
Getting Started with ModelSim

To Start a new Project and to Simulate the code, follow the following sets

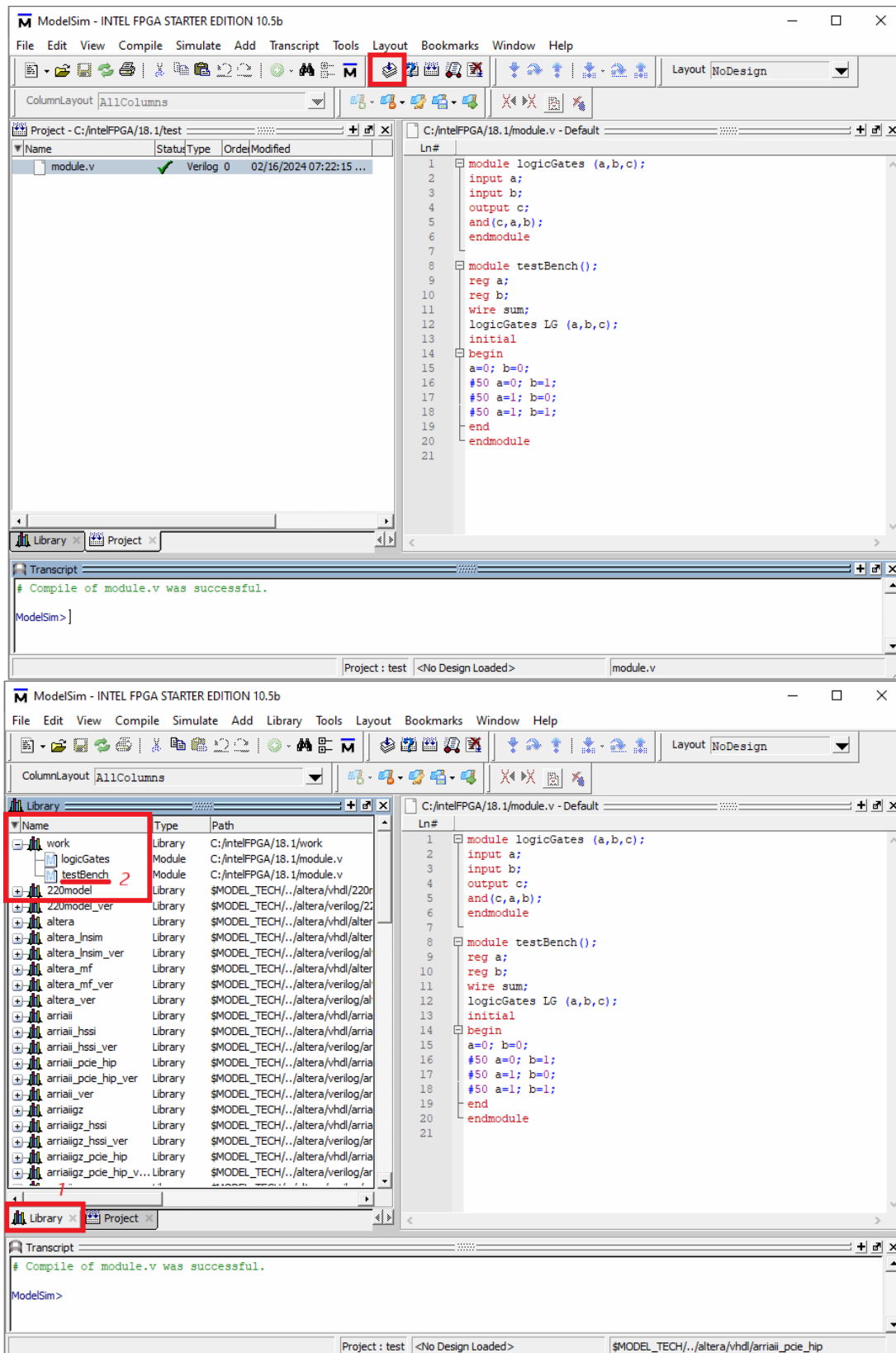


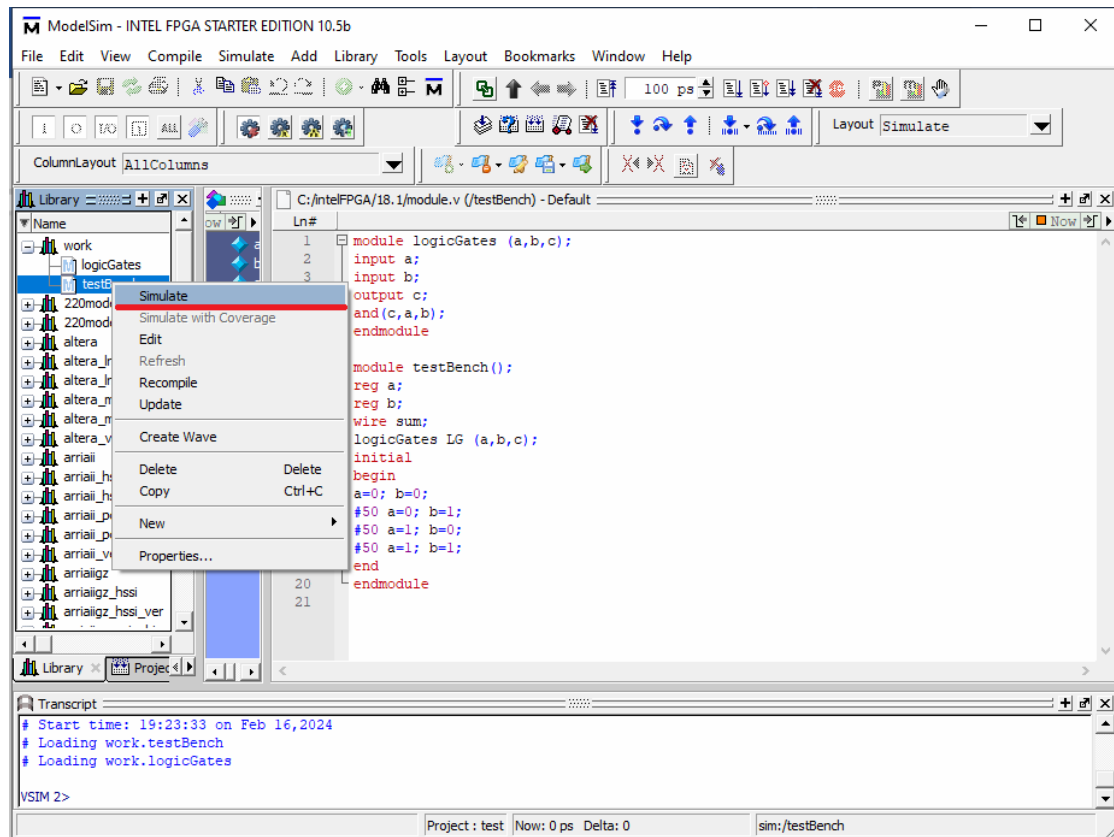


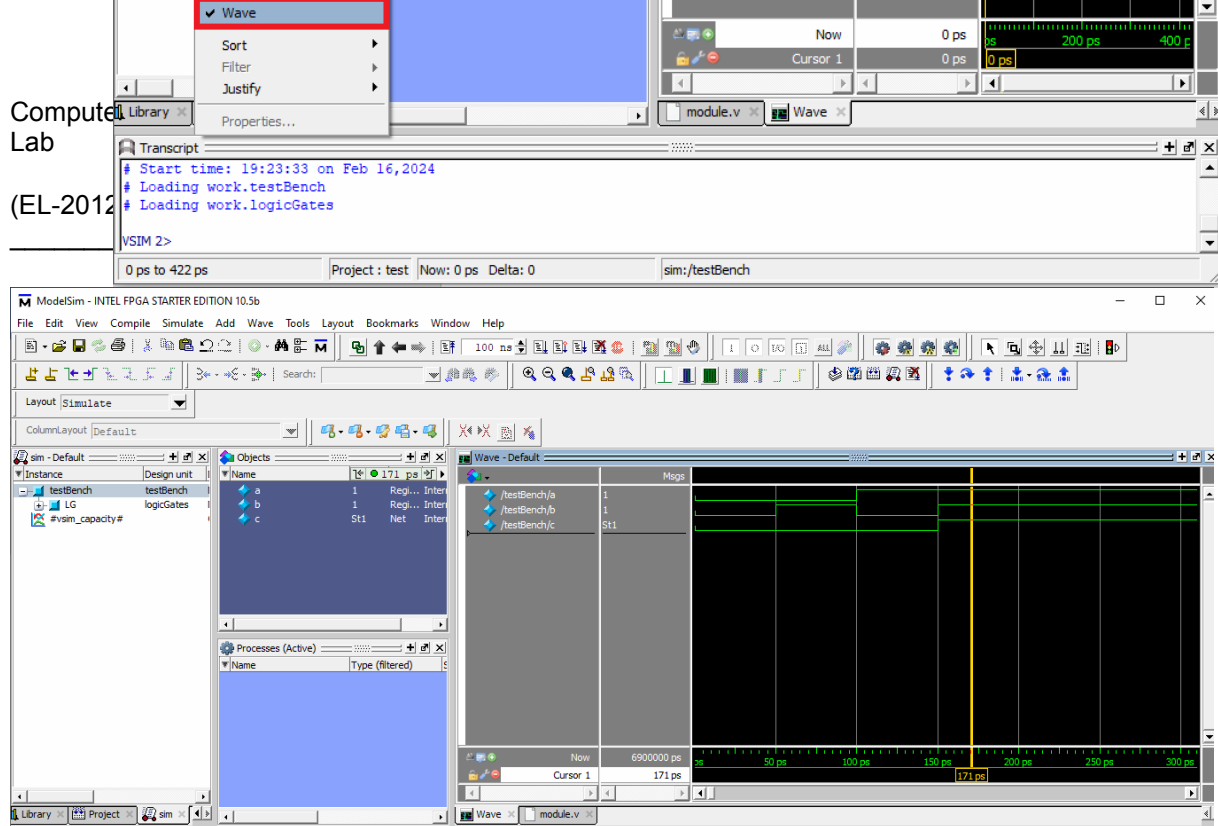




Create Basic Logic Gates



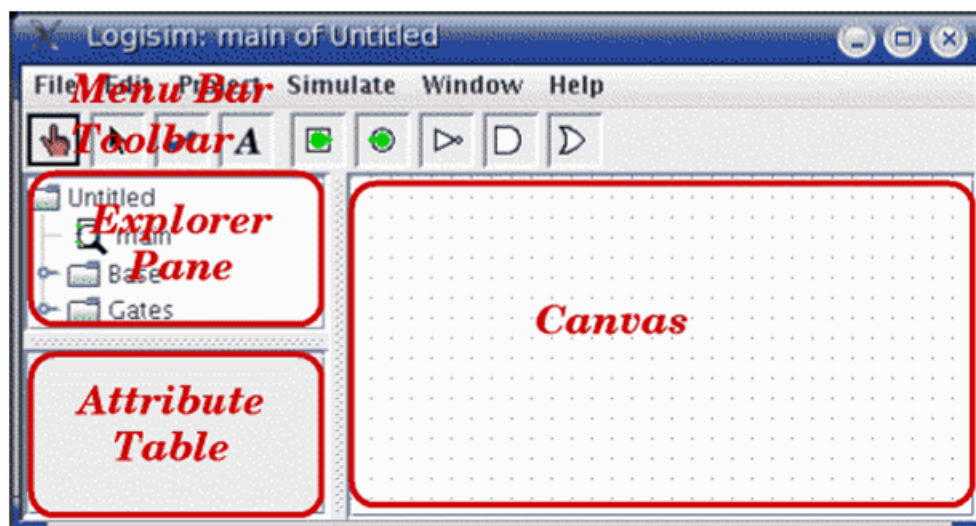




Introduction to Logisim

Logisim is an educational tool for designing and simulating digital logic circuits. With its simple toolbar interface and simulation of circuits as they are built, it is simple enough to facilitate learning the most basic concepts related to logic circuits. With the capacity to build larger circuits from smaller sub-circuits, and to draw bundles of wires with a single mouse drag, Logisim can be used (and is used) to design and simulate entire CPUs for educational purposes. Students at colleges and universities around the world use Logisim for a variety of purposes, including:

- A module in general-education computer science surveys
- A unit in sophomore-level computer organization courses
- Over a full semester in upper-division computer architecture courses

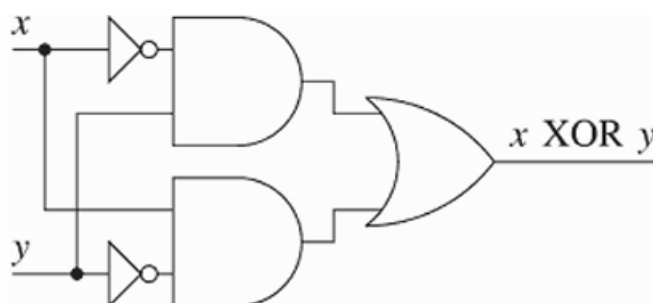


Logisim allows you to design and simulate digital circuits. It is intended as an educational tool, to help

you learn how circuits work. To practice using Logisim, let's build a XOR circuit - that is, a circuit that takes two inputs (which we'll call x and y) and outputs 1 if the inputs are the same and 0 if they are different. The following truth table illustrates.

x	y	$x \text{ XOR } y$
0	0	0
0	1	1
1	0	1
1	1	0

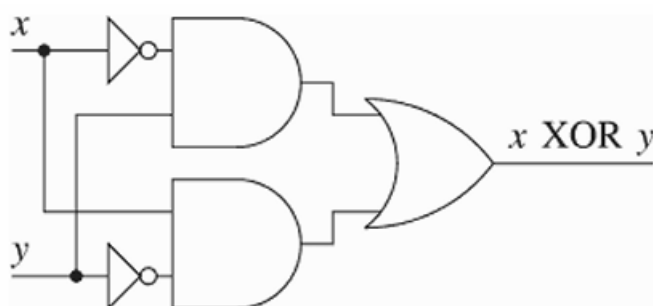
We might design such a circuit on paper.



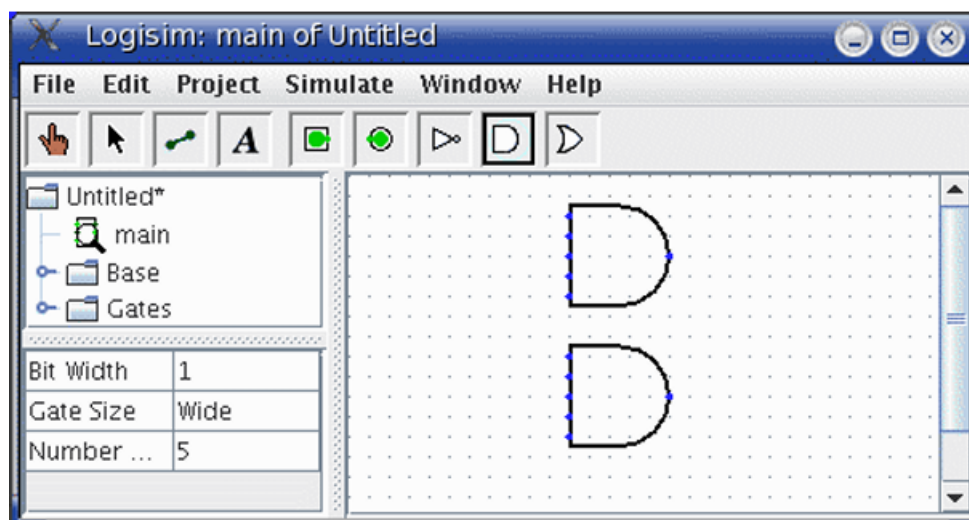
But just because it's on paper doesn't mean it's right. To verify our work, we'll draw it in Logisim and test it. As an added bonus, we'll get a circuit that's looks nicer than what you probably would draw by hand.

Step 1: Adding gates

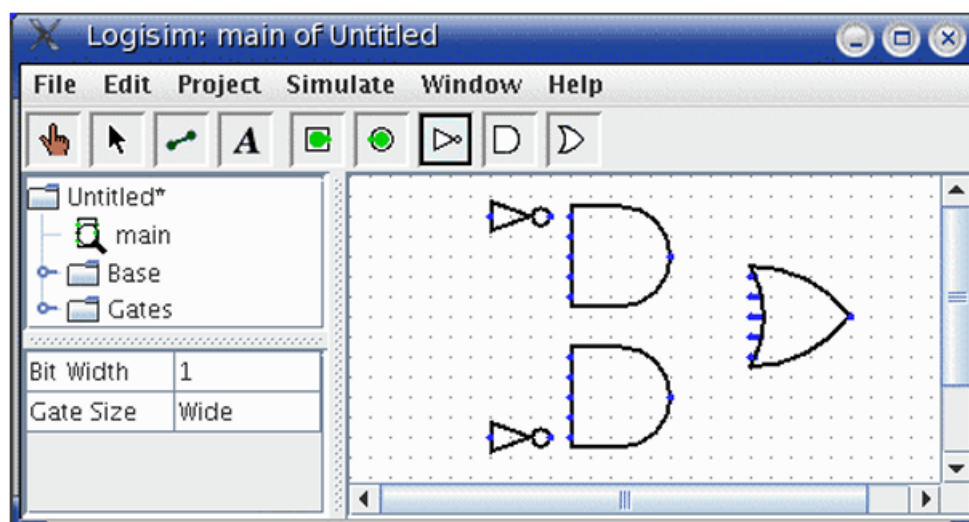
Recall that we're trying to build the following circuit in Logisim.



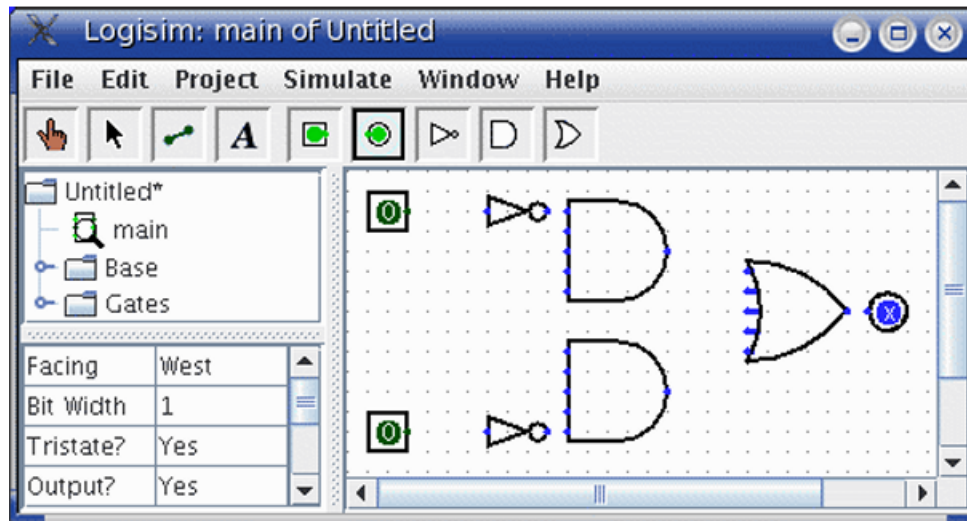
Building a circuit is easiest by inserting the gates first as a sort of skeleton for connecting wires into the circuit later. The first thing we're going to do is to add the two AND gates. Click on the AND tool in the toolbar (the next-to-last tool listed). Then click in the editing area where you want the AND gates to go. Be sure to leave plenty of room for stuff on the left.



Notice the five dots on the left side of the AND gate. These are spots where wires can be attached. It happens that we'll just use two of them for our XOR circuit; but for other circuits, you may find that having more than two wires going to an AND gate is useful. Now add the other gates. First click on the OR tool (And select the NOT tool); then click where you want it, and put those two gates into the canvas.



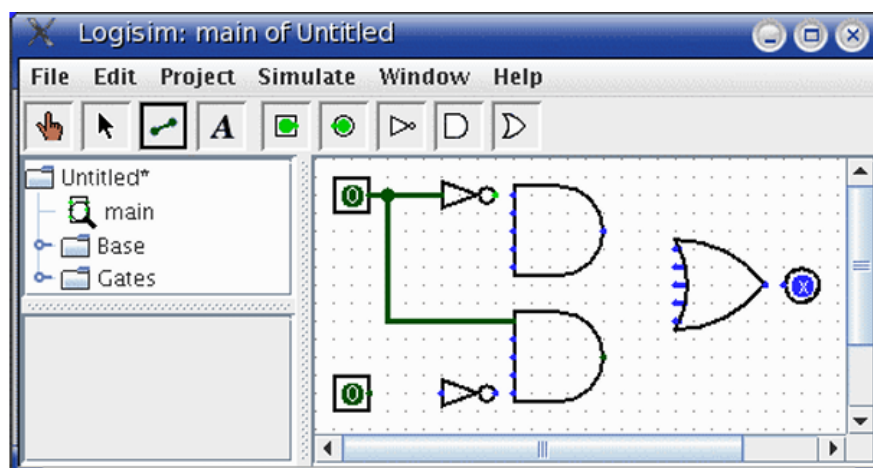
I left a little space between the NOT gates and the AND gates; if you want to, though, you can put them up against each other and save yourself the effort of drawing a wire in later. Now we want to add the two inputs x and y into the diagram. Select the input pin  and place the pins down. You should also place an output pin , next to the OR gate's output. (Again, though I'm leaving a bit of space between the OR gate and the output pin, you might choose to place them right next to each other.)



If you decide you don't like where you placed something, then you can right-click (or control-click) anything in the canvas to view a pop-up menu. Choose Delete. You can also rearrange things using the select tool .

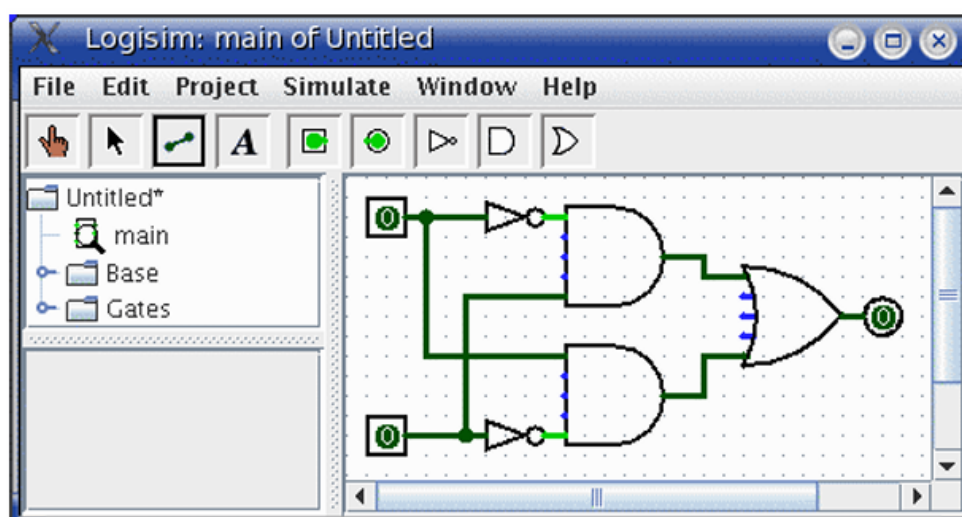
Step 2: Adding wires

After you have all the components blocked out on the canvas, you're ready to start adding wires. Select the wiring tool. Then start dragging from one position to another in the canvas area, and a wire will start to appear between the two points. Wires in Logisim must be horizontal or vertical. To connect the upper input to the NOT gate and the AND gate, then, I added three different wires.



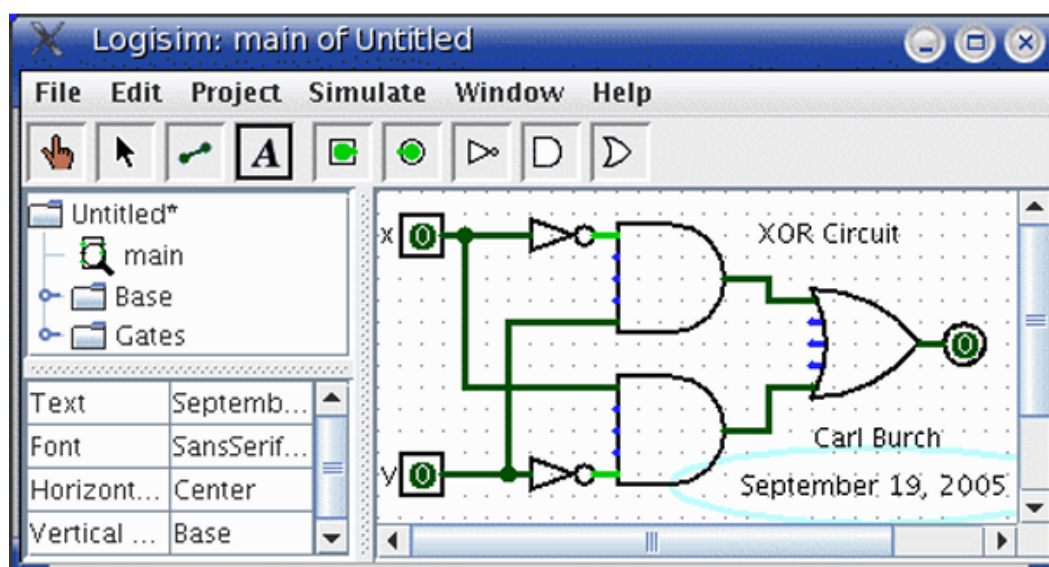
Logisim automatically connects wires to the gates and to each other. This includes automatically drawing the circle at a T intersection as above, indicating that the wires are connected. As you draw wires, you may see some blue or gray wires. Blue in Logisim indicates that the value at that point is ``unknown'', and gray indicates that the wire is not connected to anything. This is not a big deal temporarily. But by the time you finish your circuit, none of your wires should be blue or gray. (The unconnected legs of the OR gate will still be blue: That's fine.)

If you do have a blue or a gray wire after you think everything ought to be connected, then something is going wrong. It's important that you connect wires to the right places. Logisim draws little dots on the components to indicate where wires ought to connect. As you proceed, you'll see the dots turn from blue to light or dark green. Once you have all the wires connected, all of the wires you inserted will themselves be light or dark green.



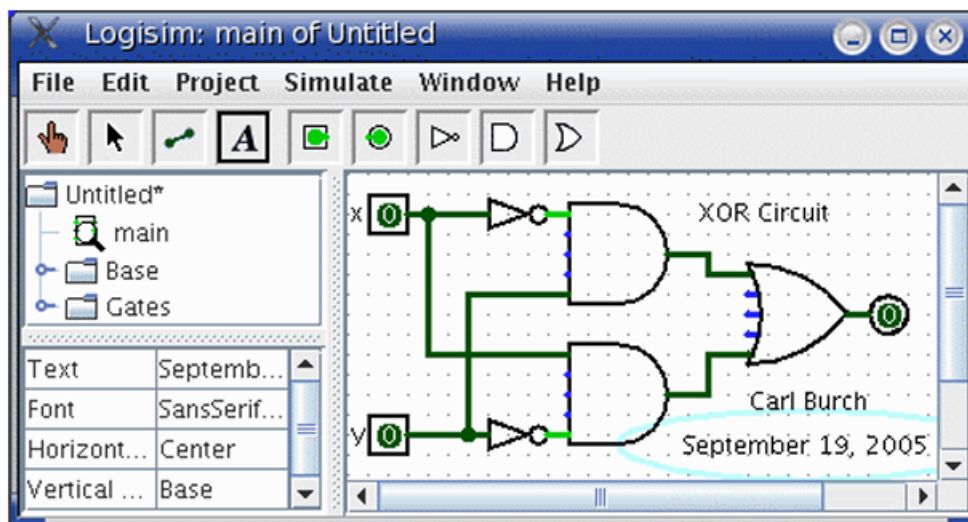
Step 3: Adding text

Adding text to the circuit isn't necessary to make it work; but if you want to show your circuit to somebody (like a teacher), then some labels help to to communicate the purpose of the different pieces of your circuit. Select the text tool (A). You can click on an input pin and start typing to give it a label. (It's better to click directly on the input pin than to click where you want the text to go, because then the label will move with the pin.) You can do the same for the output pin. Or you could just click any old place and start typing to put a label anywhere else.

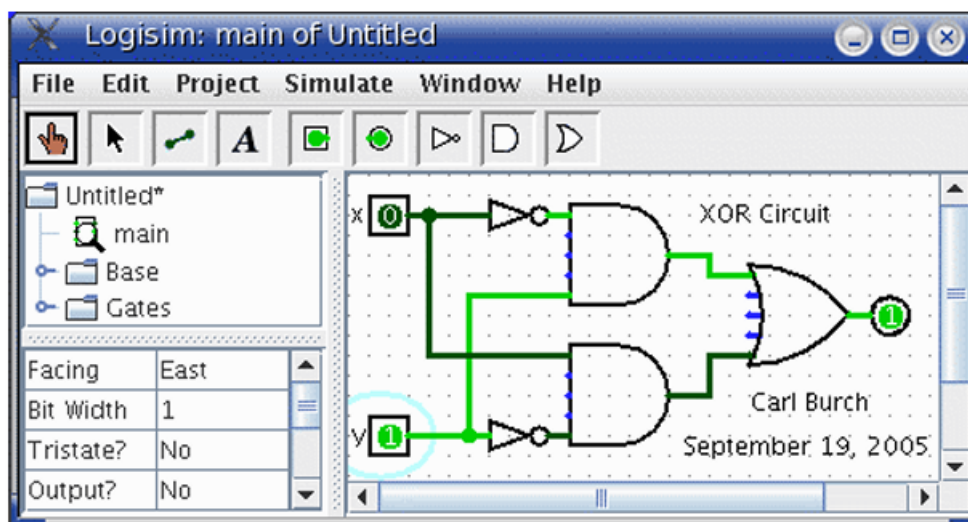


Step 4: Testing your circuit

Our final step is to test our circuit to ensure that it really does what we intended. Logisim is already simulating the circuit. Let's look again at where we were.



Note that the input pins both contain 0s; and so does the output pin. This already tells us that the circuit already computes a 0 when both inputs are 0. Now to try another combination of inputs. Select the poke tool  and start poking the inputs by clicking on them. Each time you poke an input, its value will toggle. For example, we might first poke the bottom input.



When you change the input value, Logisim will show you what values travel down the wires by drawing them light green to indicate a 1 value or dark green (almost black) to indicate a 0 value. You can also see that the output value has changed to 1. So far, we have tested the first two rows of our truth table, and the outputs (0 and 1) match the desired outputs.

x	y	$x \text{ XOR } y$
0	0	0
0	1	1
1	0	1
1	1	0

By poking the switches through different combinations, we can verify the other two rows. If they all match, then we're done: The circuit works!

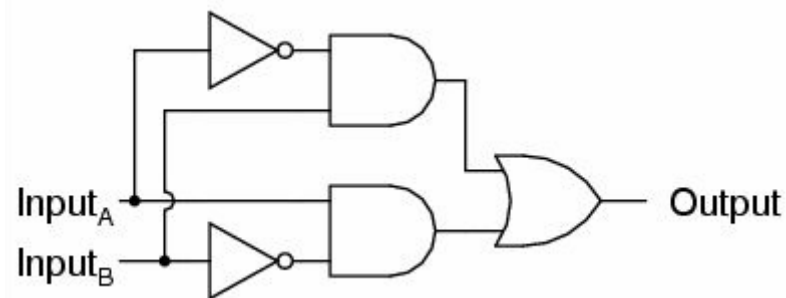
To archive your completed work, you might want to save or print your circuit. The File menu allows this, and of course it also allows you to exit Logisim. But why quit now?

Now that you are finished with the lab, you can experiment with Logisim by building your own circuits and complete the tasks. If you want to build circuits with more sophisticated features, then you should navigate through the rest of the help system to see what else you can do.

Logisim is a powerful program, allowing you to build up and test huge circuits; in this lab you just scratch the surface.

Tasks:

1. Write Verilog codes for Basic gates using structural (Gate level) modeling and verify results.
2. Write Verilog codes for Basic gates in Dataflow modeling and verify results using testbench.
3. Test the working of basic logic gates using logisim.
4. Write Verilog code for following circuit using gate level and data flow modeling and verify results using testbench.



5. Test the working of above circuit using logisim.
6. Write Verilog code to find 1's complement of four bits using gate level and data flow modeling. Verify results using testbench.